

Ansh Sharma

Roll No. 85

BE A Computer

Fashion MNIST Classification using Convolutional Neural Networks

This notebook implements a CNN to classify fashion items into 10 categories using the Fashion MNIST dataset.

Problem Statement:

Convolutional neural network (CNN): Use MNIST Fashion Dataset and create a classifier to classify fashion clothing into categories.

Dataset Description

Fashion MNIST is a dataset of Zalando's article images consisting of:

- 70,000 grayscale images (60,000 training, 10,000 testing)
- 10 categories of clothing items
- Image size: 28x28 pixels

Classes

Label	Description
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag

Label	Description
9	Ankle boot

```
In [1]: import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras import layers, models
import warnings
warnings.filterwarnings('ignore')

# Set random seed for reproducibility
np.random.seed(42)
tf.random.set_seed(42)
```

Data Loading and Preprocessing

We'll load the Fashion MNIST dataset and:

1. Normalize pixel values to [0, 1]
2. Reshape images to include channel dimension
3. Convert labels to categorical format

```
In [2]: def load_fashion_mnist():
    """Load and preprocess the Fashion MNIST dataset."""
    print("Loading Fashion MNIST dataset...")
    (X_train, y_train), (X_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()

    # Normalize pixel values to range [0, 1]
    X_train = X_train.astype('float32') / 255.0
    X_test = X_test.astype('float32') / 255.0

    # Reshape images to include channel dimension
    X_train = X_train.reshape(-1, 28, 28, 1)
    X_test = X_test.reshape(-1, 28, 28, 1)

    # Convert labels to categorical
    y_train = tf.keras.utils.to_categorical(y_train, 10)
    y_test = tf.keras.utils.to_categorical(y_test, 10)

    return (X_train, y_train), (X_test, y_test)

# Load and preprocess data
(X_train, y_train), (X_test, y_test) = load_fashion_mnist()

# Create validation split
val_samples = 5000
X_val = X_train[:val_samples]
y_val = y_train[:val_samples]
X_train = X_train[val_samples:]
y_train = y_train[val_samples:]
```

```

print(f"Training set shape: {X_train.shape}")
print(f"Validation set shape: {X_val.shape}")
print(f"Test set shape: {X_test.shape}")

# Display some sample images
plt.figure(figsize=(10, 5))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(X_train[i].reshape(28, 28), cmap='gray')
    plt.title(f'Class {np.argmax(y_train[i])}')
    plt.axis('off')
plt.tight_layout()
plt.show()

```

Loading Fashion MNIST dataset...

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz>

29515/29515 ————— **0s** 1us/step

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz>

26421880/26421880 ————— **3s** 0us/step

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz>

5148/5148 ————— **0s** 0us/step

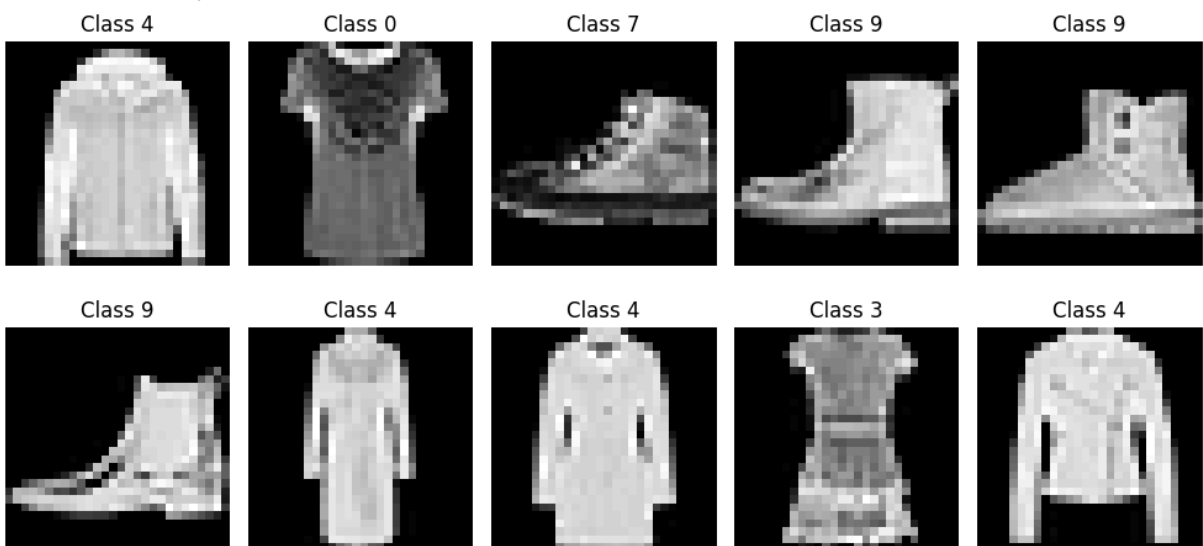
Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz>

4422102/4422102 ————— **0s** 0us/step

Training set shape: (55000, 28, 28, 1)

Validation set shape: (5000, 28, 28, 1)

Test set shape: (10000, 28, 28, 1)



Model Architecture

We'll create a CNN with:

- Multiple convolutional blocks
- Batch normalization for training stability

- MaxPooling for spatial dimension reduction
- Dropout for regularization

```
In [3]: def create_cnn_model():
        """Creates a CNN model for Fashion MNIST classification."""
        model = models.Sequential([
            # First Convolutional Block
            layers.Conv2D(32, (3, 3), activation='relu', padding='same', input_shape=(28, 28, 1)),
            layers.BatchNormalization(),
            layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
            layers.BatchNormalization(),
            layers.MaxPooling2D((2, 2)),
            layers.Dropout(0.25),

            # Second Convolutional Block
            layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
            layers.BatchNormalization(),
            layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
            layers.BatchNormalization(),
            layers.MaxPooling2D((2, 2)),
            layers.Dropout(0.25),

            # Third Convolutional Block
            layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
            layers.BatchNormalization(),
            layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
            layers.BatchNormalization(),
            layers.MaxPooling2D((2, 2)),
            layers.Dropout(0.25),

            # Dense Layers
            layers.Flatten(),
            layers.Dense(512, activation='relu'),
            layers.BatchNormalization(),
            layers.Dropout(0.5),
            layers.Dense(10, activation='softmax')
        ])

        return model

    # Create and compile model
    model = create_cnn_model()
    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Params
conv2d (Conv2D)	(None, 28, 28, 32)	
batch_normalization (BatchNormalization)	(None, 28, 28, 32)	
conv2d_1 (Conv2D)	(None, 28, 28, 32)	9
batch_normalization_1 (BatchNormalization)	(None, 28, 28, 32)	
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	
dropout (Dropout)	(None, 14, 14, 32)	
conv2d_2 (Conv2D)	(None, 14, 14, 64)	18
batch_normalization_2 (BatchNormalization)	(None, 14, 14, 64)	
conv2d_3 (Conv2D)	(None, 14, 14, 64)	36
batch_normalization_3 (BatchNormalization)	(None, 14, 14, 64)	
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 64)	
dropout_1 (Dropout)	(None, 7, 7, 64)	
conv2d_4 (Conv2D)	(None, 7, 7, 128)	73
batch_normalization_4 (BatchNormalization)	(None, 7, 7, 128)	
conv2d_5 (Conv2D)	(None, 7, 7, 128)	147
batch_normalization_5 (BatchNormalization)	(None, 7, 7, 128)	
max_pooling2d_2 (MaxPooling2D)	(None, 3, 3, 128)	
dropout_2 (Dropout)	(None, 3, 3, 128)	
flatten (Flatten)	(None, 1152)	
dense (Dense)	(None, 512)	590
batch_normalization_6 (BatchNormalization)	(None, 512)	2
dropout_3 (Dropout)	(None, 512)	
dense_1 (Dense)	(None, 10)	5

Total params: 885,738 (3.38 MB)

Trainable params: 883,818 (3.37 MB)

Non-trainable params: 1,920 (7.50 KB)

Model Training

We'll train with:

- Early stopping
- Learning rate reduction
- Batch size of 64
- Maximum 20 epochs

```
In [4]: # Training parameters
EPOCHS = 20
BATCH_SIZE = 64

# Callbacks
callbacks = [
    tf.keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=5,
        restore_best_weights=True
    ),
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.2,
        patience=3
    )
]

# Train the model
history = model.fit(
    X_train, y_train,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    validation_data=(X_val, y_val),
    callbacks=callbacks,
    verbose=1
)
```

Epoch 1/20

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
W0000 00:00:1778499282.920120 1248154 cpu_allocator_impl.cc:82] Allocation of 172480000 exceeds 10% of free system memory.

860/860 ————— **144s** 161ms/step - accuracy: 0.8122 - loss: 0.546
7 - val_accuracy: 0.8800 - val_loss: 0.3320 - learning_rate: 0.0010
Epoch 2/20

860/860 ————— **137s** 159ms/step - accuracy: 0.8787 - loss: 0.334
5 - val_accuracy: 0.9064 - val_loss: 0.2574 - learning_rate: 0.0010
Epoch 3/20

860/860 ————— **135s** 157ms/step - accuracy: 0.8966 - loss: 0.284
9 - val_accuracy: 0.9198 - val_loss: 0.2232 - learning_rate: 0.0010
Epoch 4/20

860/860 ————— **135s** 157ms/step - accuracy: 0.9064 - loss: 0.256
7 - val_accuracy: 0.9226 - val_loss: 0.2152 - learning_rate: 0.0010
Epoch 5/20

860/860 ————— **135s** 157ms/step - accuracy: 0.9116 - loss: 0.241
5 - val_accuracy: 0.9090 - val_loss: 0.2575 - learning_rate: 0.0010
Epoch 6/20

860/860 ————— **135s** 157ms/step - accuracy: 0.9181 - loss: 0.225
2 - val_accuracy: 0.9198 - val_loss: 0.2159 - learning_rate: 0.0010
Epoch 7/20

860/860 ————— **145s** 160ms/step - accuracy: 0.9241 - loss: 0.209
0 - val_accuracy: 0.9092 - val_loss: 0.2417 - learning_rate: 0.0010
Epoch 8/20

860/860 ————— **135s** 157ms/step - accuracy: 0.9389 - loss: 0.168
1 - val_accuracy: 0.9398 - val_loss: 0.1616 - learning_rate: 2.0000e-04
Epoch 9/20

860/860 ————— **138s** 160ms/step - accuracy: 0.9447 - loss: 0.151
6 - val_accuracy: 0.9396 - val_loss: 0.1675 - learning_rate: 2.0000e-04
Epoch 10/20

860/860 ————— **135s** 157ms/step - accuracy: 0.9493 - loss: 0.140
7 - val_accuracy: 0.9400 - val_loss: 0.1648 - learning_rate: 2.0000e-04
Epoch 11/20

860/860 ————— **137s** 160ms/step - accuracy: 0.9509 - loss: 0.132
6 - val_accuracy: 0.9428 - val_loss: 0.1697 - learning_rate: 2.0000e-04
Epoch 12/20

860/860 ————— **137s** 159ms/step - accuracy: 0.9562 - loss: 0.119
1 - val_accuracy: 0.9414 - val_loss: 0.1615 - learning_rate: 4.0000e-05
Epoch 13/20

860/860 ————— **135s** 157ms/step - accuracy: 0.9574 - loss: 0.115
4 - val_accuracy: 0.9440 - val_loss: 0.1599 - learning_rate: 4.0000e-05
Epoch 14/20

860/860 ————— **137s** 160ms/step - accuracy: 0.9589 - loss: 0.112
7 - val_accuracy: 0.9428 - val_loss: 0.1602 - learning_rate: 4.0000e-05
Epoch 15/20

860/860 ————— **135s** 157ms/step - accuracy: 0.9597 - loss: 0.110
1 - val_accuracy: 0.9440 - val_loss: 0.1598 - learning_rate: 4.0000e-05
Epoch 16/20

860/860 ————— **137s** 160ms/step - accuracy: 0.9593 - loss: 0.109
9 - val_accuracy: 0.9442 - val_loss: 0.1620 - learning_rate: 4.0000e-05
Epoch 17/20

860/860 ————— **135s** 157ms/step - accuracy: 0.9610 - loss: 0.106
6 - val_accuracy: 0.9450 - val_loss: 0.1636 - learning_rate: 4.0000e-05
Epoch 18/20

860/860 ————— **136s** 158ms/step - accuracy: 0.9613 - loss: 0.104
1 - val_accuracy: 0.9452 - val_loss: 0.1630 - learning_rate: 4.0000e-05
Epoch 19/20

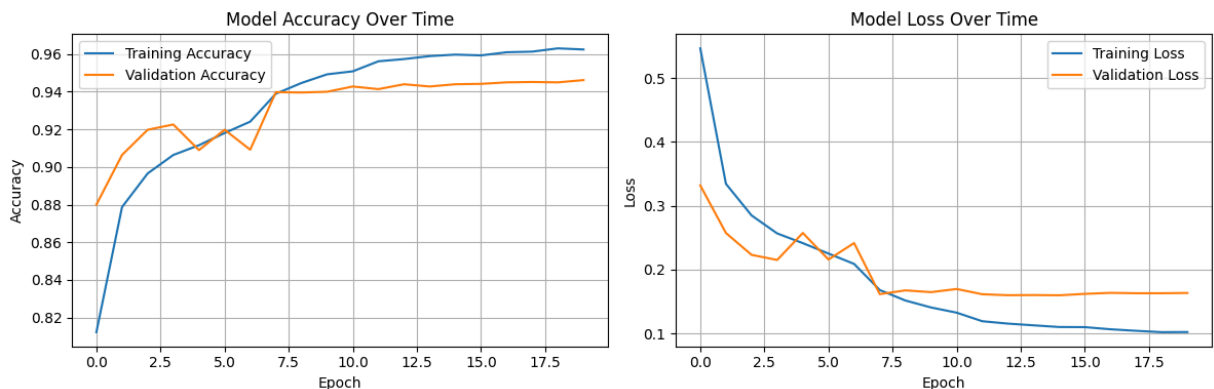
860/860 ————— **135s** 157ms/step - accuracy: 0.9631 - loss: 0.102
0 - val_accuracy: 0.9450 - val_loss: 0.1630 - learning_rate: 8.0000e-06

Epoch 20/20

860/860 ————— **136s** 158ms/step - accuracy: 0.9625 - loss: 0.102
2 - val_accuracy: 0.9462 - val_loss: 0.1634 - learning_rate: 8.0000e-06

Training History Visualization

```
In [5]: def plot_training_history(history):  
        """Plot the training and validation accuracy/loss curves."""  
        plt.figure(figsize=(12, 4))  
  
        # Accuracy plot  
        plt.subplot(1, 2, 1)  
        plt.plot(history.history['accuracy'], label='Training Accuracy')  
        plt.plot(history.history['val_accuracy'], label='Validation Accuracy')  
        plt.title('Model Accuracy Over Time')  
        plt.xlabel('Epoch')  
        plt.ylabel('Accuracy')  
        plt.legend()  
        plt.grid(True)  
  
        # Loss plot  
        plt.subplot(1, 2, 2)  
        plt.plot(history.history['loss'], label='Training Loss')  
        plt.plot(history.history['val_loss'], label='Validation Loss')  
        plt.title('Model Loss Over Time')  
        plt.xlabel('Epoch')  
        plt.ylabel('Loss')  
        plt.legend()  
        plt.grid(True)  
  
        plt.tight_layout()  
        plt.show()  
  
        plot_training_history(history)
```



Model Evaluation and Sample Predictions

```
In [6]: def plot_sample_predictions(model, X_test, y_test, num_samples=10):  
        """Plot sample predictions with actual and predicted labels."""  
        class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',  
                        'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
```



```

predictions = model.predict(X_test[:num_samples])

plt.figure(figsize=(15, 4))
for i in range(num_samples):
    plt.subplot(2, 5, i + 1)
    plt.imshow(X_test[i].reshape(28, 28), cmap='gray')
    true_label = class_names[np.argmax(y_test[i])]
    pred_label = class_names[np.argmax(predictions[i])]
    color = 'green' if true_label == pred_label else 'red'
    plt.title(f'True: {true_label}\nPred: {pred_label}', color=color)
    plt.axis('off')
plt.tight_layout()
plt.show()

# Evaluate on test set
test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f"\nTest Accuracy: {test_accuracy:.4f}")

# Plot sample predictions
print("\nGenerating sample predictions...")
plot_sample_predictions(model, X_test, y_test)

# Plot confusion matrix
from sklearn.metrics import confusion_matrix
import seaborn as sns

y_pred = np.argmax(model.predict(X_test), axis=1)
y_true = np.argmax(y_test, axis=1)
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()

```

Test Accuracy: 0.9390

Generating sample predictions...

1/1 ————— 0s 213ms/step

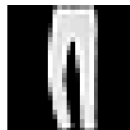
True: Ankle boot
Pred: Ankle boot



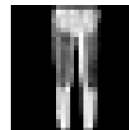
True: Pullover
Pred: Pullover



True: Trousers
Pred: Trousers



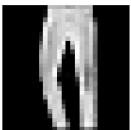
True: Trousers
Pred: Trousers



True: Shirt
Pred: Shirt



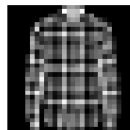
True: Trousers
Pred: Trousers



True: Coat
Pred: Coat



True: Shirt
Pred: Shirt



True: Sandal
Pred: Sandal



True: Sneaker
Pred: Sneaker



313/313 ————— 8s 24ms/step

